
Design Patterns

I. Les design patterns utilisés:

1. Singleton Pattern :

Le pattern Singleton a été utilisé pour gérer le service de notifications dans l'application TaskFlow.

Ce pattern garantit qu'une seule instance du service de notification existe dans tout le système, ce qui permet une gestion centralisée et cohérente des notifications envoyées aux utilisateurs.

- **Pourquoi utilisé :**

- ✓ Éviter la création de plusieurs services de notification.
- ✓ Assurer une gestion unique des messages et alertes.
- ✓ Réduire la consommation des ressources.

- **Exemple dans TaskFlow :**

Lorsqu'une tâche est assignée à un membre, le système utilise une seule instance du NotificationService pour envoyer la notification.

- **Code Python :**

```
class ServiceNotification:  
    _instance = None  
    def __new__(cls):  
        if cls._instance is None:
```

```
cls._instance = super(ServiceNotification,
cls).__new__(cls)
return cls._instance
def envoyer_notification(self, message):
print(f"Notification : {message}")
notif1 = ServiceNotification()
notif2 = ServiceNotification()
notif1.envoyer_notification("Tâche assignée")
print(notif1 is notif2)
```

2. Observer Pattern

Le pattern Observer a été utilisé afin de notifier automatiquement les utilisateurs lorsqu'un changement se produit sur une tâche.

- **Pourquoi utilisé :**

- ✓ Faciliter la communication entre composants.
- ✓ Rendre le système plus dynamique.
- ✓ Respecter le principe de faible couplage.

- **Exemple dans TaskFlow :**

Quand l'administrateur attribue une tâche à un membre, une notification est envoyée automatiquement sans modifier directement les autres composants.

- **Code Python :**

```
class Membre:
def __init__(self, nom):
self.nom = nom
def update(self, message):
```

```

print(f"{self.nom} reçoit : {message}")
class Tache:
def __init__(self, titre):
self.titre = titre
self.observers = []
def ajouter_observer(self, observer):
self.observers.append(observer)
def notifier(self, message):
for observer in self.observers:
observer.update(message)
m1 = Membre("Sara")
m2 = Membre("Soumaya")
tache = Tache("Créer UML")
tache.ajouter_observer(m1)
tache.ajouter_observer(m2)
tache.notifier("Une nouvelle tâche vous a été
assignée")

```

3. Factory Pattern

Le pattern Factory a été utilisé pour créer les différents types d'utilisateurs dans l'application.

Au lieu de créer directement les objets avec new, la création est centralisée dans une classe Factory.

- **Pourquoi utilisé :**

- ✓ Simplifier la création des objets.
- ✓ Faciliter l'ajout de nouveaux types d'utilisateurs.
- ✓ Améliorer la maintenabilité du code.

- **Exemple dans TaskFlow :**

La classe UserFactory permet de créer :

un Administrateur

un Membre simple

selon le type demandé

- **Code Python :**

```
class Utilisateur:
    def afficher_role(self):
        pass
class Admin(Utilisateur):
    def afficher_role(self):
        print("Je suis un Administrateur")
class Membre(Utilisateur):
    def afficher_role(self):
        print("Je suis un Membre")
class FactoryUtilisateur:
    @staticmethod
    def creer_utilisateur(type_user):
        if type_user == "admin":
            return Admin()
        elif type_user == "membre":
            return Membre()
        else:
            raise ValueError("Type d'utilisateur invalide")
user1 = FactoryUtilisateur.creer_utilisateur("admin")
user2 =
FactoryUtilisateur.creer_utilisateur("membre")
user1.afficher_role()
user2.afficher_role()
```